

Visual Language State Machine Robot*

Elias Goldsztejn, Dan Rouven Suissa, Ronen Brafman

Department of Computer Science, Ben Gurion University of the Negev, Beer Sheva, Israel

Keywords: Cognitive Robotics, Robot and Multi-Robot Systems, Task Planning and Execution, Vision and Perception

Abstract: Autonomous mobile robots in human environments must balance efficiency with social compliance and safety. While Vision-Language Models (VLMs) enable semantic scene understanding, existing frameworks often lack dynamic reconfiguration in response to changing scenarios, gestures, and hazards. We present zero-shot VLM integration without fine-tuning using a modular state-machine architecture that dynamically adapts navigation, gesture-triggered transitions, and safety-critical overrides. Experiments show robust gesture recognition, object detection, and scenario adaptability, with seamless behavior switching at expected transition points.

1 INTRODUCTION

Effective interaction with complex environments and humans is a key challenge for autonomous robotic systems. Vision-Language Models (VLMs) are promising tools for addressing these challenges thanks to their semantically rich perception and decision-making capabilities (Liu et al., 2024; Jeong et al., 2024). In this paper, we utilize VLMs to control and coordinate multiple off-the-shelf behaviors, allowing them to adapt dynamically to environmental changes, object detection, and human gestures. Unlike prior work relying on fine-tuning (Yang et al., 2024; Li et al., 2023) or end-to-end policies (Wang et al., 2024), our modular system requires no training and uses zero-shot VLM outputs to adapt its behavior.

Our system leverages the VLM to interpret scenes, gestures, and hazards, dynamically adjusting behavior without per-task training. For navigation, the robot adapts its algorithm parameters based on scene understanding; for Human Robot Interaction (HRI), it interprets gestures to trigger behaviors like following; for safety, it recognizes hazards and either navigates more cautiously or hands control to a human operator. These behaviors are synchronized by a state-based controller informed by the VLM output. Rather than aiming for exhaustive coverage, we showcase how combining VLMs with control structures enables rich, adaptive robot behavior in diverse settings.

*This work was supported by ISF Grant 1651/19, the Helmsley Charitable Trust through the ABC Robotics Center of Ben-Gurion University, and the Lynn and William Frankel Center for Computer Science.

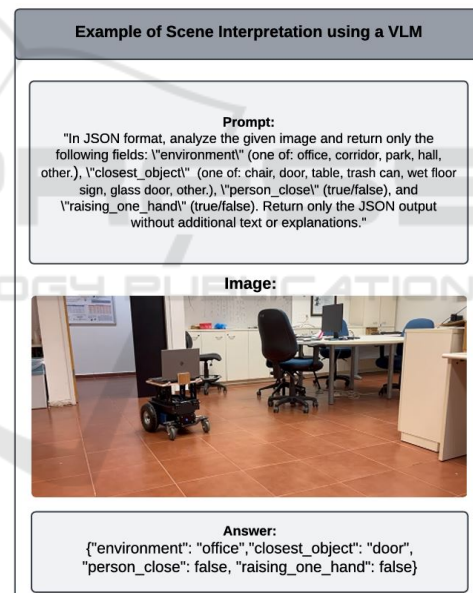


Figure 1: Example of scene description using Llama-Vision 11b in an office environment. The VLM accurately found that the environment is an office, the closest object is a door, and that there are no people close. The robot dynamically adjusts its inflation radius for smoother door passage and reduces its velocity accordingly.

Specifically, we propose a zero-shot approach to high-level robot control using a VLM combined with a state machine, without additional task-specific fine-tuning. The VLM is used both to perform context-based parameter tuning and to drive the higher-level state-machine controller. This modular approach can easily accommodate additional be-

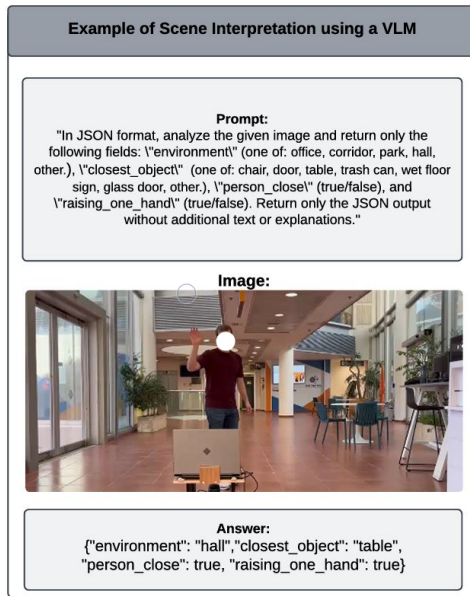


Figure 2: Example of scene description using Llama-Vision 11b. The VLM accurately found that the environment is a hall, and a person is raising his hand. Because the person is raising his hand, the robot starts following him.

haviors. In summary, our contribution is a unified system that integrates and enhances diverse adaptive skills through zero-shot VLM perception, using a modular state-machine framework, validated through empirical evaluation. Our system is open-sourced, and our code is freely available online: <https://github.com/vlsmr/Visual-Language-State-Machine-Robot>

2 BACKGROUND AND RELATED WORK

Visual Language Models (VLMs). Vision-Language Models (VLMs) (Li et al., 2025) combine computer vision and natural language processing to process and relate visual and textual data. As multimodal systems, they enable foundational tasks such as image captioning, VQA, and image-text retrieval. A typical VLM uses specialized encoders to establish semantic connections across modalities. Recent efforts apply VLMs to robotics: (Li et al., 2023) introduced RoboFlamingo for manipulation, and (Song et al., 2024) proposed VLM-Social-Nav for socially aware navigation. (Cheng et al., 2024) also developed NaVILA, integrating VLM perception with locomotion for legged robots. These works demonstrate the VLM’s capacity to enhance flexibility and context-awareness in robotic systems.

Adaptive Navigation using VLMs. Effective navigation requires dynamic parameter adjustment based on environmental elements like corridors or doorways. VLMs generate high-level semantic representations (Liu et al., 2024; Huang et al., 2023) that integrate perception with real-time decision-making. VLM-GroNav (Elnoor et al., 2024) estimates terrain properties using VLM and proprioception to dynamically adjust traversability. BehAV (Weerakoon et al., 2024) utilizes zero-shot scene understanding, allowing robots to follow complex instructions and respect constraints. Unlike fixed-policy pipelines (Yokoyama et al., 2024; Elnoor et al., 2024; Weerakoon et al., 2024), our system leverages VLM outputs to adjust a state machine, enabling more flexible and adaptive high-level behavior.

Gesture Recognition using VLMs. Human gesture understanding is critical for HRI, yet traditional methods often rely on intensive tracking (e.g., HMMs (Chen et al., 2003)), large labeled datasets (Yu et al., 2022), or wearable sensors (Cheng et al., 2021). VLM-based methods offer a flexible alternative. Recent work, such as GIRAF (Lin et al., 2023), combines vision models with language prompts to interpret gestures and infer intent. GestLLM (Kobzarev et al., 2025) uses large language models and feature extraction (MediaPipe) to recognize diverse gestures without training. Our work moves beyond simple recognition by integrating gestures into a state-machine framework that directly triggers behavioral changes, such as initiating a “follow” mode.

Object and Hazard Detection. Accurate object recognition is vital for robot navigation. While classical methods like YOLO offer speed (Terven et al., 2023), VLMs provide greater flexibility and contextual understanding (Zhang et al., 2021; Li et al., 2024). VLMs also excel in hazard detection. INSIGHT (Chen et al., 2025) is a hierarchical VLM framework for autonomous driving that integrates inputs to detect unpredictable scenarios and improve safety. We utilize the VLM to detect both general contexts and specific dangerous objects, incorporating this information into a state-machine-governed safety protocol for an immediate switch to human control.

Distinction from Prior Work. Our approach integrates continuous scene understanding and adaptive control into a unified modular framework, extending capabilities beyond monolithic methods like OpenFMNav (Kuang et al., 2024). Compared to VLM-based systems—VLM-Social-Nav (Song et al., 2024), BehAV (Weerakoon et al., 2024), and NaVILA (Cheng et al., 2024)—our system offers critical advancements. Where these often focus on navigation or fixed policies, our system leverages zero-shot

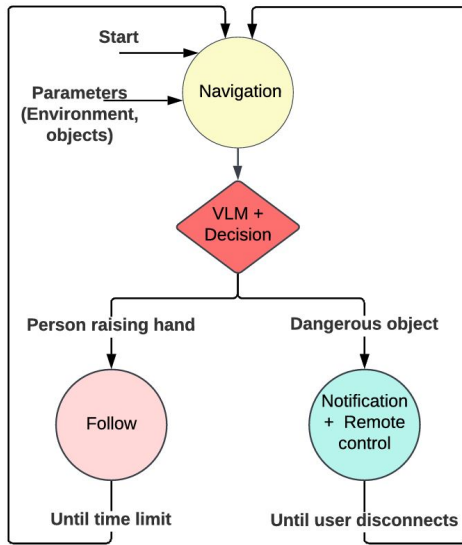


Figure 3: State diagram. The VLM analyzes images and provides descriptions, while a decision module adjusts the robot's states accordingly.

VLM perception within a hierarchical state machine for dynamic parameter adaptation based on real-time environmental context. Crucially, our framework supports seamless, gesture-triggered mode transitions and immediate safety-driven overrides without requiring fine-tuning or retraining. This inherent flexibility distinguishes our work from approaches that rely on dedicated training or rigid operational pipelines.

3 APPROACH

3.1 Robot

We use a RoverZero robot (Fig. 5) with a top-mounted computer providing real-time visualization. The state machine (external computer) sends messages to the robot. The robot operates using ROS (Quigley et al., 2009) and employs the Dynamic Window Approach (DWA) (Fox et al., 1997) for Navigation. In this state, the state machine sets maximum speed and inflation radius parameters while the robot autonomously follows goals. In Follow and Remote states, the state machine directly controls velocity.

3.2 State-Machine (Master)

The state machine uses the transitions package (Bronger et al., 2024) and communicates via Python's multiprocessing shared memory (van Rossum and the Python Development Team, 2024). This ensures modularity by maintaining independent spaces for

Initialize: History buffer $H \leftarrow \emptyset$

Input : JSON from VLM; environment dict E ; hazardous set O_h ; object dict O

Output : $\langle state \rangle \langle message \rangle$ to the robot

while true do

```

    data  $\leftarrow$  GetJSONFromVLM()
    append data to  $H$ 
    (person_close, raising_hand, object, env)  $\leftarrow$  MajorityVote( $H$ )
    max_speed  $\leftarrow E[env]$  // retrieve max speed for environment
    inflation_radius  $\leftarrow O[object]$  // inflation radius from object dict
    if person_close  $\wedge$  raising_hand then
        state  $\leftarrow$  FOLLOW
    else
        if object  $\in O_h$  then
            state  $\leftarrow$  REMOTE
        else
            state  $\leftarrow$  NAVIGATION
        end
    end
end

```

Algorithm 1: State Machine Decision Process.

each module. The machine exchanges commands based on a JSON file generated from parsed VLM descriptions of images captured at F Hz. A rule-based system determines the state (Fig. 3, Algorithm 1). See Fig. 6 for the robot's view in different states. We implemented a stabilization mechanism using majority voting over the last k entries in a history buffer. This ensures outputs are selected no more frequently than every 2 seconds and no less than every 10 seconds.

Since the VLM's response frequency varies, we implemented a stabilization mechanism using majority voting over the last k entries in a history buffer. This ensures outputs are selected no more frequently than every 2 seconds and no less than every 10 seconds. The value of k is adapted based on the VLM's response rate.

3.3 VLM Error Correction + Parser

We use Llama-Vision-11b via the GROQCloud API, achieving approximately 2 Hz throughput. We continuously upload images with a structured prompt querying: (1) Environment type, (2) Nearby objects, (3) Person presence, and (4) Hand gestures. See Figs. 1 and 2. Responses are retrieved in JSON. The module handles errors, reattempts uploads, and cleans the

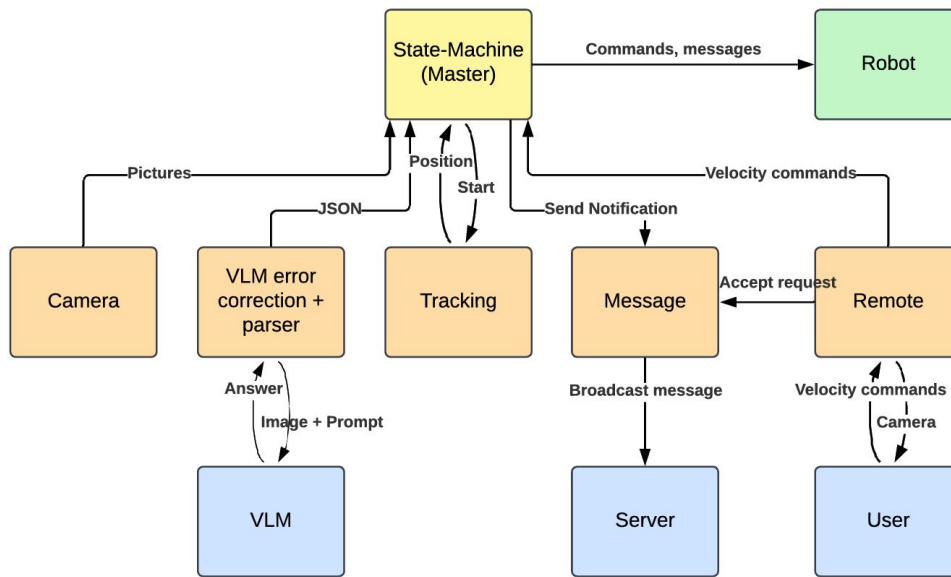


Figure 4: The diagram illustrates the interaction between the various modules and their core messages. The State-Machine (Master) communicates with independent modules to determine the appropriate state, and subsequently sends commands to the robot.

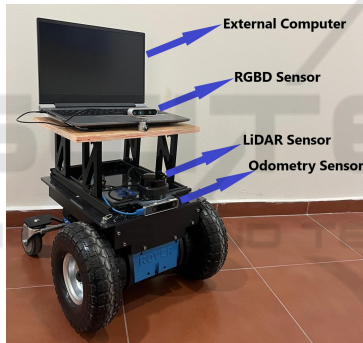


Figure 5: The system setup includes a RoverZero Robot, an external computer, and a suite of sensors: RGBD, LiDAR, and odometry.

output (lowercasing, filtering) before sending the processed JSON to the State-Machine.

3.4 Tracking

This module activates when a nearby person raises a hand (detected by VLM). Tracking is performed using Detectron2 (Wu et al., 2019), which runs locally and is faster than the VLM for identification. We compute the center of mass (x, y, z) for each person using segmentation and RGB-D data. At approx. 10 Hz, we assign positions by identifying the closest updated center of mass. To handle occlusions, the system stores the last known position and waits a short interval. Proximity constraints stop the robot if the person is too close. We convert the (x, z) position into velocity commands using linear interpola-

tion: $\text{linear_vel} = 1$, $\text{angular_vel} = k \cdot \frac{x}{z}$. See Algorithm 2.

3.5 Message Notification Application

Even state-of-the-art navigation can falter in hazardous situations, so a readily activatable fallback is essential.

In our implementation, VLM-detected objects are validated by the state machine against a predefined list of hazards (e.g., wet-floor sign, glass door); on a match, the Message module is invoked.

We developed a notification application for Android, using the Firebase (Firebase, 2011) protocol to broadcast messages to all app users. The Message module connects to a dedicated Ktor (Ktor, 2018) server and sends the message: "Request to Help Robot: $\langle \text{dangerous} - \text{object} \rangle$ ". All users receive this notification.

The robot then halts and waits for the first user to establish a connection. Upon connection, the robot sends the message: "Got Help, Thanks!" The user then remotely controls the robot to move it away from the dangerous object. See Fig. 7

3.6 User Remote Control of Robot

When a user receives the notification, they can choose to assist the robot. The first user to do so gains real-time access to the robot's camera feed and can send commands to move the robot.

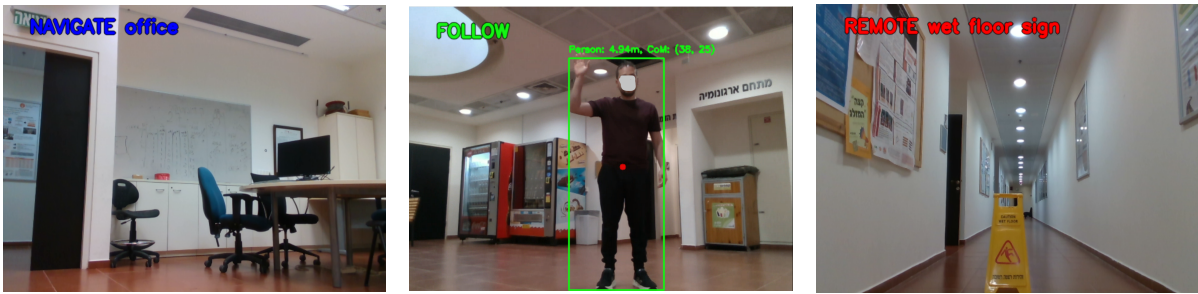


Figure 6: Screenshots from the robot’s camera, showing its point of view in different states.

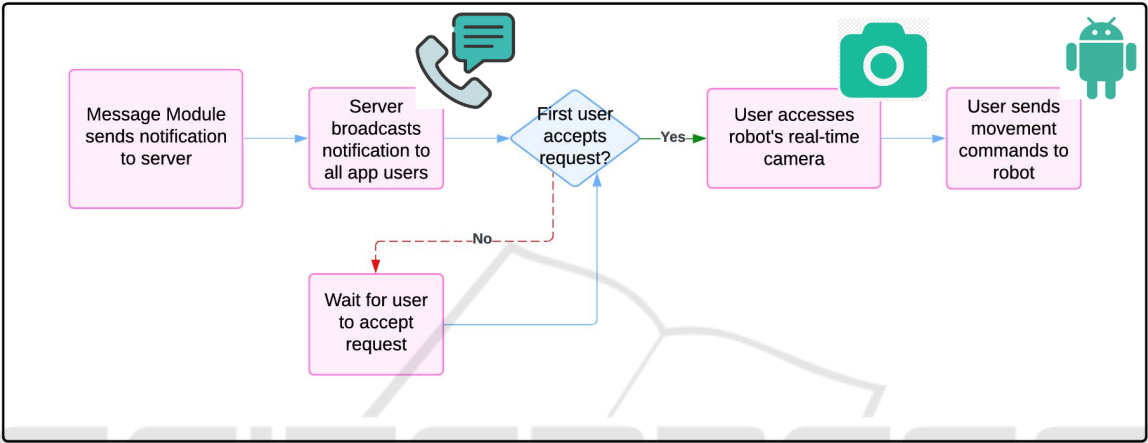


Figure 7: This diagram illustrates the remote control protocol, including the notification service and the user connection to the robot.

We implemented the remote control functionality using a Wi-Fi connection with a socket protocol and observed a slight connection latency of approximately 0.5 seconds.

This approach is well-suited for our scenario, as the robot operates within the university campus, which has a robust and expansive Wi-Fi network.

4 EXPERIMENTS

Implementation runs on a PC (13th Gen i7, 32GB RAM, RTX 4070) and a RoverZero robot (Intel NUC i7, RPLiDAR A3, RealSense TD256 & D435). We used Llama-Vision 11b.

We first evaluate VLM recognition capabilities, followed by autonomous navigation trials in two circuits to measure expected switches against ground-truth annotations.

4.1 Recognition

We measured recognition distance and time (averaged over 10 trials) for Follow mode (raised hand) and Re-

mote mode (wet floor sign), as summarized in Table 1. Environment recognition precision, calculated over 2-minute traversals, is detailed in Table 2.

Both gestures and hazards were reliably detected within 2s (including network latency) at distances up to 10m and 12.5m, respectively. Environment recognition achieved 100% accuracy for Office and Corridor settings. However, accuracy dropped for Hall (86%) and Park (78%) due to visual ambiguities, where specific camera angles resembled other environments.

4.2 State-Machine

We designed two test circuits containing specific hazards and gesture zones. Using DWA for autonomous navigation, we conducted 10 trials per circuit. Performance was evaluated against manually annotated ground truth by recording the robot-to-object distance at the moment of each mode switch.

To assess the behavior-switching performance of our VLM-based system, we employ the following metrics:

Input : RGB-D image stream; Detectron2 model

Output: Velocity commands (linear_vel, angular_vel)

```

if tracking activated then
  start timer  $T \leftarrow 30$  s
  while tracking active do
    run Detectron2
    extract people centers of mass  $(x,y,z)$ 
    match detected persons to previous positions
    if tracking lost  $> 1$  s or  $T$  expired then
      exit tracking
    end
    if too close then
      stop robot
    end
    else
      compute and send velocity commands
      linear_vel  $\leftarrow 1$ 
      angular_vel  $\leftarrow k \cdot \frac{x}{z}$ 
    end
  end
end
end

```

Algorithm 2: Person Tracking Module.

Table 1: Minimum and maximum distance and recognition time for Remote and Follow modes.

	Remote	Follow
Min. Distance (m)	0.5	2.0
Max. Distance (m)	12.5	10.0
Recognition Time (s)	1.8 ± 0.3	1.6 ± 0.1

- **Behavior Alignment Score (BAS):** The percentage of correct behavior transitions—i.e., instances where the robot switches to the correct mode at the expected location—out of all expected transition points. This score indicates how well the system aligns with intended behavior (Jacoff et al., 2002; Chen et al., 2023). Higher is better.
- **Transition Latency (L_t):** The mean absolute distance between the ground-truth trigger location (object or person) and the robot’s actual switching location, quantifying system responsiveness (Jacoff et al., 2002; Norton et al., 2022). Lower is better.
- **False Positive Rate (FPR):** The number of erroneous behavior activations per minute of operation, capturing switching precision (Norton et al., 2022). Lower is better.

Table 2: Recognition accuracy in different environments.

Navigation Environment	Office	Hall	Corridor	Park
Recognition Environment	100%	86%	100%	78%

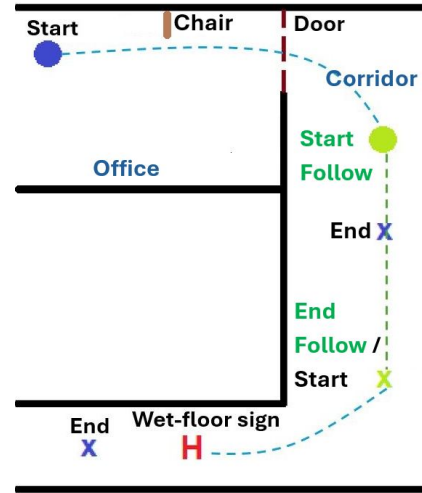


Figure 8: Circuit A: The blue line represents Navigation mode, and the green line indicates Follow mode. Red “H” markers denote hazardous objects.

Circuit A is described in Fig. 8. The robot started in an office (Navigation mode), encountered a chair, adjusted its inflation parameter to cross a door into a corridor, then turned and detected a person raising a hand, switching to Follow mode for up to 5 meters. It reverted to Navigation mode for 1 meter before detecting a wet floor sign and switching to Remote mode.

In circuit B, the robot began in a park-like central courtyard within a building (Navigation mode), observed stairs, and entered Remote mode upon detecting a glass door. It then crossed into a corridor and switched to Follow mode upon detecting a person raising a hand.

The results are summarized in Table 3. In Circuit A, all environments and objects were correctly recognized, resulting in successful execution of all required behavior switches ($BAS = 100\%$). Although object locations (L_t) varied across trials, they were consistently recognized before the corresponding switches. A false positive rate (FPR) of 0.10 was observed due to occasional misclassification of the corridor as an office.

In Circuit B, a few expected behavior switches were missed ($BAS = 93\%$), primarily due to the park being misclassified as an office in several trials—likely because it was located at a building entrance. Additionally, the glass door was not recognized in one trial, leading to an aborted experiment

Table 3: Behavior assessment across different circuits. BAS: Behavior Accuracy Score, L_t : Latency to switch, FPR: False Positive Rate.

Metric	Circuit A (Office → Corridor)	Circuit B (Park → Corridor)
BAS (%)	100	93
L_t chair (m)	0.0 ± 0.0	–
L_t door (m)	1.35 ± 0.24	–
L_t wet sign (Remote) (m)	0.60 ± 0.44	–
L_t person (Follow) (m)	1.8 ± 0.35	0.78 ± 0.55
L_t stairs (m)	–	0.0 ± 0.0
L_t glass door (Remote) (m)	–	1.06 ± 1.08
FPR (/min)	0.10	0.48



Figure 9: The robot stops when it detects a hazardous object, showing how visual language models help handle unseen cases. Left: a red “do not pass” string. Right: a ladder.

due to a potential collision. Despite these issues, most experiments showed correct recognition of environments and objects at the expected locations.

Transition latency was $< 2m$ at all sites in both circuits, meaning the robot switched no more than $2m$ after the expected trigger locations.

The system robustly recognizes environments, objects, and gestures in real-world settings, enabling accurate behavior switching across circuits. Navigation is adapted to context—e.g., moving faster in parks or reducing inflation radius to pass through doors—resulting in more efficient navigation. Minor failures were due to ambiguous or low-visibility situations.

4.3 Open-Ended Recognition with VLM

A key advantage of VLMs is their open-world adaptability. To test this, we introduced a general “hazardous_object_detected” prompt to trigger Remote

mode, bypassing predefined object lists. Experiments with unseen hazards—specifically a ladder and a barrier string—resulted in successful state transitions (see Fig. 9), demonstrating the system’s capacity to recognize and react to novel situations dynamically.

5 CONCLUSION

In this work, we demonstrate how a zero-shot system can leverage image descriptions provided by a VLM to dynamically adapt a robot’s operational modes. Our approach integrates three key components for mobile navigation: (1) adaptive navigation, (2) gesture recognition, and (3) hazardous object detection. By deploying this system on a real robot, we validate its effectiveness and highlight the essential components required for practical implementation. This work offers a unified framework for high-level robot control using zero-shot learning, allowing for reliable decision-making without the need for task-specific training. Although we expect VLM capabilities to continually improve, their successful performance within our system indicates that they already offer much utility for controlling autonomous robots.

6 LIMITATIONS

A key limitation of this work is the lack of temporal awareness. The system only queries prompts based on the current situation and ignores the past. While this is effective for clear, short-term decisions, it may be less suitable for long-term planning. Integrating temporal logic could help address this limitation.

REFERENCES

- Bronger, T. et al. (2024). transitions: A lightweight, object-oriented finite state machine implementation in python.
- Chen, D., Zhang, Z., Liu, Y., and Yang, X. T. (2025). Insight: Enhancing autonomous driving safety through vision-language models on context-aware hazard detection and edge case evaluation. *arXiv e-prints*, pages arXiv–2502.
- Chen, F.-S., Fu, C.-M., and Huang, C.-L. (2003). Hand gesture recognition using a real-time tracking method and hidden markov models. *Image and vision computing*, 21(8):745–758.
- Chen, H. L., Hendrikx, B., Torta, E., Bruyninckx, H., and van de Molengraft, R. (2023). Behavior adaptation for mobile robots via semantic map compositions of

- constraint-based controllers. *Frontiers in Robotics and AI*, 10:917637.
- Cheng, A.-C., Ji, Y., Yang, Z., Gongye, Z., Zou, X., Kautz, J., Bryik, E., Yin, H., Liu, S., and Wang, X. (2024). Navila: Legged robot vision-language-action model for navigation. *arXiv preprint arXiv:2412.04453*.
- Cheng, Y., Li, G., Yu, M., Jiang, D., Yun, J., Liu, Y., Liu, Y., and Chen, D. (2021). Gesture recognition based on surface electromyography-feature image. *Concurrency and Computation: Practice and Experience*, 33(6):e6051.
- Elnoor, M., Weerakoon, K., Seneviratne, G., Xian, R., Guan, T., Jaffar, M. K. M., Rajagopal, V., and Manocha, D. (2024). Robot navigation using physically grounded vision-language models in outdoor environments. *arXiv preprint arXiv:2409.20445*.
- Firestore (2011). Firestore documentation. <https://firebase.google.com/docs>. Accessed: 2025-03-13.
- Fox, D., Burgard, W., and Thrun, S. (1997). The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, 4(1):23–33.
- Huang, C., Mees, O., Zeng, A., and Burgard, W. (2023). Visual language maps for robot navigation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10608–10615. IEEE.
- Jacoff, A., Messina, E., and Evans, J. (2002). Performance evaluation of autonomous mobile robots. *Industrial Robot: An International Journal*, 29(3):259–267.
- Jeong, H., Lee, H., Kim, C., and Shin, S. (2024). A survey of robot intelligence with large language models. *Applied Sciences*, 14(19):8868.
- Kobzarev, O., Lykov, A., and Tsetserukou, D. (2025). Gestllm: Advanced hand gesture interpretation via large language models for human-robot interaction. *arXiv preprint arXiv:2501.07295*.
- Ktor (2018). Ktor documentation. <https://ktor.io/docs/>. Accessed: 2025-03-13.
- Kuang, Y., Lin, H., and Jiang, M. (2024). Openfmnav: Towards open-set zero-shot object navigation via vision-language foundation models. *arXiv preprint arXiv:2402.10670*.
- Li, X., Liu, M., Zhang, H., Yu, C., Xu, J., Wu, H., Cheang, C., Jing, Y., Zhang, W., Liu, H., et al. (2023). Vision-language foundation models as effective robot imitators. *arXiv preprint arXiv:2311.01378*.
- Li, Z., Wu, X., Du, H., Liu, F., Nghiem, H., and Shi, G. (2025). A survey of state of the art large vision language models: Alignment, benchmark, evaluations and challenges. *arXiv preprint arXiv:2501.02189*.
- Li, Z., Xiang, Z., West, J., and Khoshelham, K. (2024). From open vocabulary to open world: Teaching vision language models to detect novel objects. *arXiv preprint arXiv:2411.18207*.
- Lin, L.-H., Cui, Y., Hao, Y., Xia, F., and Sadigh, D. (2023). Gesture-informed robot assistance via foundation models. In *7th Annual Conference on Robot Learning*.
- Liu, S., Zhang, J., Gao, R. X., Wang, X. V., and Wang, L. (2024). Vision-language model-driven scene understanding and robotic object manipulation. In *2024 IEEE 20th International Conference on Automation Science and Engineering (CASE)*, pages 21–26. IEEE.
- Norton, A., Admoni, H., Crandall, J., Fitzgerald, T., Gautam, A., Goodrich, M., Saretsky, A., Scheutz, M., Simmons, R., Steinfeld, A., et al. (2022). Metrics for robot proficiency self-assessment and communication of proficiency in human-robot teams. *ACM Transactions on Human-Robot Interaction (THRI)*, 11(3):1–38.
- Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., and Ng, A. (2009). Ros: an open-source robot operating system. In *ICRA Workshop on Open Source Software*, volume 3.
- Song, D., Liang, J., Payandeh, A., Xiao, X., and Manocha, D. (2024). Socially aware robot navigation through scoring using vision-language models. *arXiv e-prints*, pages arXiv–2404.
- Terven, J., Córdova-Esparza, D.-M., and Romero-González, J.-A. (2023). A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas. *Machine learning and knowledge extraction*, 5(4):1680–1716.
- van Rossum, G. and the Python Development Team (2024). *Python multiprocessing shared memory module*. Python.
- Wang, Z., Shen, R., and Stadie, B. (2024). Solving robotics problems in zero-shot with vision-language models. *arXiv preprint arXiv:2407.19094*.
- Weerakoon, K., Elnoor, M., Seneviratne, G., Rajagopal, V., Arul, S. H., Liang, J., Jaffar, M. K. M., and Manocha, D. (2024). Behav: Behavioral rule guided autonomy using vlms for robot navigation in outdoor scenes. *arXiv preprint arXiv:2409.16484*.
- Wu, Y., Kirillov, A., Massa, F., Lo, W.-Y., and Girshick, R. (2019). Detectron2. <https://github.com/facebookresearch/detectron2>.
- Yang, J., Mark, M. S., Vu, B., Sharma, A., Bohg, J., and Finn, C. (2024). Robot fine-tuning made easy: Pre-training rewards and policies for autonomous real-world reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4804–4811. IEEE.
- Yokoyama, N., Ha, S., Batra, D., Wang, J., and Bucher, B. (2024). Vlfm: Vision-language frontier maps for zero-shot semantic navigation. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 42–48. IEEE.
- Yu, J., Qin, M., and Zhou, S. (2022). Dynamic gesture recognition based on 2d convolutional neural network and feature fusion. *Scientific Reports*, 12(1):4345.
- Zhang, P., Li, X., Hu, X., Yang, J., Zhang, L., Wang, L., Choi, Y., and Gao, J. (2021). Vinvl: Revisiting visual representations in vision-language models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5579–5588.